

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

А. В. Шевчук

студент группы ПО-13

Проверил:

А. Д. Кулик

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Цель работы: приобрести практические навыки разработки API и баз данных.

Общее задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

database.py:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "postgresql+psycopg2://postgres:123@localhost:5432/education_db"

engine = create_engine(DATABASE_URL)
SESSION_LOCAL = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()
```

models.py:

```
# pylint: disable=too-few-public-methods

from sqlalchemy import Column, Date, ForeignKey, Integer, String
from sqlalchemy.orm import relationship

from .database import Base

class EducationalInstitution(Base):
    """Educational institution table."""

    __tablename__ = "educational_institutions"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(150), nullable=False)
    city = Column(String(100), nullable=False)
    address = Column(String(200), nullable=False)
    institution_type = Column(String(50), nullable=False)

    departments = relationship(
        "Department",
        back_populates="institution",
        cascade="all, delete",
    )

class Department(Base):
    """Department table."""

    __tablename__ = "departments"
```

```

id = Column(Integer, primary_key=True, index=True)
institution_id = Column(
    Integer,
    ForeignKey("educational_institutions.id"),
    nullable=False,
)
name = Column(String(150), nullable=False)
phone = Column(String(30), nullable=False)

institution = relationship("EducationalInstitution", back_populates="departments")
programs = relationship(
    "Program",
    back_populates="department",
    cascade="all, delete",
)
courses = relationship(
    "Course",
    back_populates="department",
    cascade="all, delete",
)

class Program(Base):
    """Program table."""

    __tablename__ = "programs"

    id = Column(Integer, primary_key=True, index=True)
    department_id = Column(Integer, ForeignKey("departments.id"), nullable=False)
    name = Column(String(150), nullable=False)
    degree_level = Column(String(50), nullable=False)
    duration_years = Column(Integer, nullable=False)

    department = relationship("Department", back_populates="programs")
    students = relationship(
        "Student",
        back_populates="program",
        cascade="all, delete",
    )

class Student(Base):
    """Student table."""

    __tablename__ = "students"

    id = Column(Integer, primary_key=True, index=True)
    program_id = Column(Integer, ForeignKey("programs.id"), nullable=False)
    full_name = Column(String(150), nullable=False)
    birth_date = Column(Date, nullable=False)
    email = Column(String(120), unique=True, nullable=False)
    enrollment_year = Column(Integer, nullable=False)

    program = relationship("Program", back_populates="students")
    enrollments = relationship(
        "Enrollment",
        back_populates="student",

```

```

        cascade="all, delete",
    )

class Course(Base):
    """Course table."""

    __tablename__ = "courses"

    id = Column(Integer, primary_key=True, index=True)
    department_id = Column(Integer, ForeignKey("departments.id"), nullable=False)
    name = Column(String(150), nullable=False)
    credits = Column(Integer, nullable=False)
    semester = Column(Integer, nullable=False)

    department = relationship("Department", back_populates="courses")
    enrollments = relationship(
        "Enrollment",
        back_populates="course",
        cascade="all, delete",
    )

class Enrollment(Base):
    """Enrollment table."""

    __tablename__ = "enrollments"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.id"), nullable=False)
    course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
    enrollment_date = Column(Date, nullable=False)
    grade = Column(String(10), nullable=False)

    student = relationship("Student", back_populates="enrollments")
    course = relationship("Course", back_populates="enrollments")

```

schemas.py:

```

# pylint: disable=too-few-public-methods

from datetime import date

from pydantic import BaseModel, ConfigDict

class EducationalInstitutionCreate(BaseModel):
    """Schema for creating an educational institution."""

    name: str
    city: str
    address: str
    institution_type: str

class EducationalInstitutionResponse(EducationalInstitutionCreate):
    """Schema for educational institution response."""

    id: int
    model_config = ConfigDict(from_attributes=True)

```

```
class DepartmentCreate(BaseModel):
    """Schema for creating a department."""

    institution_id: int
    name: str
    phone: str

class DepartmentResponse(DepartmentCreate):
    """Schema for department response."""

    id: int
    model_config = ConfigDict(from_attributes=True)

class ProgramCreate(BaseModel):
    """Schema for creating a program."""

    department_id: int
    name: str
    degree_level: str
    duration_years: int

class ProgramResponse(ProgramCreate):
    """Schema for program response."""

    id: int
    model_config = ConfigDict(from_attributes=True)

class StudentCreate(BaseModel):
    """Schema for creating a student."""

    program_id: int
    full_name: str
    birth_date: date
    email: str
    enrollment_year: int

class StudentResponse(StudentCreate):
    """Schema for student response."""

    id: int
    model_config = ConfigDict(from_attributes=True)

class CourseCreate(BaseModel):
    """Schema for creating a course."""

    department_id: int
    name: str
    credits: int
    semester: int

class CourseResponse(CourseCreate):
    """Schema for course response."""

    id: int
    model_config = ConfigDict(from_attributes=True)
```

```

class EnrollmentCreate(BaseModel):
    """Schema for creating an enrollment."""

    student_id: int
    course_id: int
    enrollment_date: date
    grade: str

class EnrollmentResponse(EnrollmentCreate):
    """Schema for enrollment response."""

    id: int
    model_config = ConfigDict(from_attributes=True)

```

main.py:

```

from typing import Generator

from fastapi import Depends, FastAPI, HTTPException
from sqlalchemy.orm import Session

from .database import Base, SESSION_LOCAL, engine
from .models import Course, Department, EducationalInstitution, Enrollment, Program, Student
from .schemas import (
    CourseCreate,
    CourseResponse,
    DepartmentCreate,
    DepartmentResponse,
    EducationalInstitutionCreate,
    EducationalInstitutionResponse,
    EnrollmentCreate,
    EnrollmentResponse,
    ProgramCreate,
    ProgramResponse,
    StudentCreate,
    StudentResponse,
)

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Educational Institutions API")

def get_db() -> Generator[Session, None, None]:
    """Provide database session."""
    db = SESSION_LOCAL()
    try:
        yield db
    finally:
        db.close()

@app.post("/institutions/", response_model=EducationalInstitutionResponse)
def create_institution(
    institution: EducationalInstitutionCreate,
    db: Session = Depends(get_db),
) -> EducationalInstitution:

```

```

"""Create educational institution."""
new_item = EducationalInstitution(**institution.model_dump())
db.add(new_item)
db.commit()
db.refresh(new_item)
return new_item

@app.get("/institutions/", response_model=list[EducationalInstitutionResponse])
def get_institutions(db: Session = Depends(get_db)) -> list[EducationalInstitution]:
    """Get all educational institutions."""
    return db.query(EducationalInstitution).all()

@app.put("/institutions/{institution_id}", response_model=EducationalInstitutionResponse)
def update_institution(
    institution_id: int,
    institution: EducationalInstitutionCreate,
    db: Session = Depends(get_db),
) -> EducationalInstitution:
    """Update educational institution."""
    found_item = db.query(EducationalInstitution).filter(EducationalInstitution.id == institution_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Institution not found")

    for key, value in institution.model_dump().items():
        setattr(found_item, key, value)

    db.commit()
    db.refresh(found_item)
    return found_item

@app.delete("/institutions/{institution_id}")
def delete_institution(
    institution_id: int,
    db: Session = Depends(get_db),
) -> dict[str, str]:
    """Delete educational institution."""
    found_item = db.query(EducationalInstitution).filter(EducationalInstitution.id == institution_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Institution not found")

    db.delete(found_item)
    db.commit()
    return {"message": "Institution deleted"}

@app.post("/departments/", response_model=DepartmentResponse)
def create_department(
    department: DepartmentCreate,
    db: Session = Depends(get_db),
) -> Department:
    """Create department."""
    institution = (
        db.query(EducationalInstitution).filter(EducationalInstitution.id == department.institution_id).first()
    )
    if institution is None:
        raise HTTPException(status_code=404, detail="Institution not found")

```

```

new_item = Department(**department.model_dump())
db.add(new_item)
db.commit()
db.refresh(new_item)
return new_item

```

```

@app.get("/departments/", response_model=list[DepartmentResponse])
def get_departments(db: Session = Depends(get_db)) -> list[Department]:
    """Get all departments."""
    return db.query(Department).all()

```

```

@app.put("/departments/{department_id}", response_model=DepartmentResponse)
def update_department(
    department_id: int,
    department: DepartmentCreate,
    db: Session = Depends(get_db),
) -> Department:
    """Update department."""
    found_item = db.query(Department).filter(Department.id == department_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Department not found")

    for key, value in department.model_dump().items():
        setattr(found_item, key, value)

    db.commit()
    db.refresh(found_item)
    return found_item

```

```

@app.delete("/departments/{department_id}")
def delete_department(
    department_id: int,
    db: Session = Depends(get_db),
) -> dict[str, str]:
    """Delete department."""
    found_item = db.query(Department).filter(Department.id == department_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Department not found")

    db.delete(found_item)
    db.commit()
    return {"message": "Department deleted"}

```

```

@app.post("/programs/", response_model=ProgramResponse)
def create_program(
    program: ProgramCreate,
    db: Session = Depends(get_db),
) -> Program:
    """Create educational program."""
    department = db.query(Department).filter(Department.id == program.department_id).first()
    if department is None:
        raise HTTPException(status_code=404, detail="Department not found")

    new_item = Program(**program.model_dump())
    db.add(new_item)
    db.commit()

```



```
db.refresh(new_item)
return new_item
```

```
@app.get("/programs/", response_model=list[ProgramResponse])
def get_programs(db: Session = Depends(get_db)) -> list[Program]:
    """Get all educational programs."""
    return db.query(Program).all()
```

```
@app.put("/programs/{program_id}", response_model=ProgramResponse)
def update_program(
    program_id: int,
    program: ProgramCreate,
    db: Session = Depends(get_db),
) -> Program:
    """Update educational program."""
    found_item = db.query(Program).filter(Program.id == program_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Program not found")

    for key, value in program.model_dump().items():
        setattr(found_item, key, value)

    db.commit()
    db.refresh(found_item)
    return found_item
```

```
@app.delete("/programs/{program_id}")
def delete_program(
    program_id: int,
    db: Session = Depends(get_db),
) -> dict[str, str]:
    """Delete educational program."""
    found_item = db.query(Program).filter(Program.id == program_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Program not found")

    db.delete(found_item)
    db.commit()
    return {"message": "Program deleted"}
```

```
@app.post("/students/", response_model=StudentResponse)
def create_student(
    student: StudentCreate,
    db: Session = Depends(get_db),
) -> Student:
    """Create student."""
    program = db.query(Program).filter(Program.id == student.program_id).first()
    if program is None:
        raise HTTPException(status_code=404, detail="Program not found")

    new_item = Student(**student.model_dump())
    db.add(new_item)
    db.commit()
    db.refresh(new_item)
    return new_item
```

```

@app.get("/students/", response_model=list[StudentResponse])
def get_students(db: Session = Depends(get_db)) -> list[Student]:
    """Get all students."""
    return db.query(Student).all()

@app.put("/students/{student_id}", response_model=StudentResponse)
def update_student(
    student_id: int,
    student: StudentCreate,
    db: Session = Depends(get_db),
) -> Student:
    """Update student."""
    found_item = db.query(Student).filter(Student.id == student_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Student not found")

    for key, value in student.model_dump().items():
        setattr(found_item, key, value)

    db.commit()
    db.refresh(found_item)
    return found_item

@app.delete("/students/{student_id}")
def delete_student(
    student_id: int,
    db: Session = Depends(get_db),
) -> dict[str, str]:
    """Delete student."""
    found_item = db.query(Student).filter(Student.id == student_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Student not found")

    db.delete(found_item)
    db.commit()
    return {"message": "Student deleted"}

@app.post("/courses/", response_model=CourseResponse)
def create_course(
    course: CourseCreate,
    db: Session = Depends(get_db),
) -> Course:
    """Create course."""
    department = db.query(Department).filter(Department.id == course.department_id).first()
    if department is None:
        raise HTTPException(status_code=404, detail="Department not found")

    new_item = Course(**course.model_dump())
    db.add(new_item)
    db.commit()
    db.refresh(new_item)
    return new_item

@app.get("/courses/", response_model=list[CourseResponse])
def get_courses(db: Session = Depends(get_db)) -> list[Course]:
    """Get all courses."""

```

```

return db.query(Course).all()

@app.put("/courses/{course_id}", response_model=CourseResponse)
def update_course(
    course_id: int,
    course: CourseCreate,
    db: Session = Depends(get_db),
) -> Course:
    """Update course."""
    found_item = db.query(Course).filter(Course.id == course_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Course not found")

    for key, value in course.model_dump().items():
        setattr(found_item, key, value)

    db.commit()
    db.refresh(found_item)
    return found_item

@app.delete("/courses/{course_id}")
def delete_course(
    course_id: int,
    db: Session = Depends(get_db),
) -> dict[str, str]:
    """Delete course."""
    found_item = db.query(Course).filter(Course.id == course_id).first()
    if found_item is None:
        raise HTTPException(status_code=404, detail="Course not found")

    db.delete(found_item)
    db.commit()
    return {"message": "Course deleted"}

@app.post("/enrollments/", response_model=EnrollmentResponse)
def create_enrollment(
    enrollment: EnrollmentCreate,
    db: Session = Depends(get_db),
) -> Enrollment:
    """Create enrollment."""
    student = db.query(Student).filter(Student.id == enrollment.student_id).first()
    course = db.query(Course).filter(Course.id == enrollment.course_id).first()

    if student is None:
        raise HTTPException(status_code=404, detail="Student not found")
    if course is None:
        raise HTTPException(status_code=404, detail="Course not found")

    new_item = Enrollment(**enrollment.model_dump())
    db.add(new_item)
    db.commit()
    db.refresh(new_item)
    return new_item

@app.get("/enrollments/", response_model=list[EnrollmentResponse])
def get_enrollments(db: Session = Depends(get_db)) -> list[Enrollment]:

```

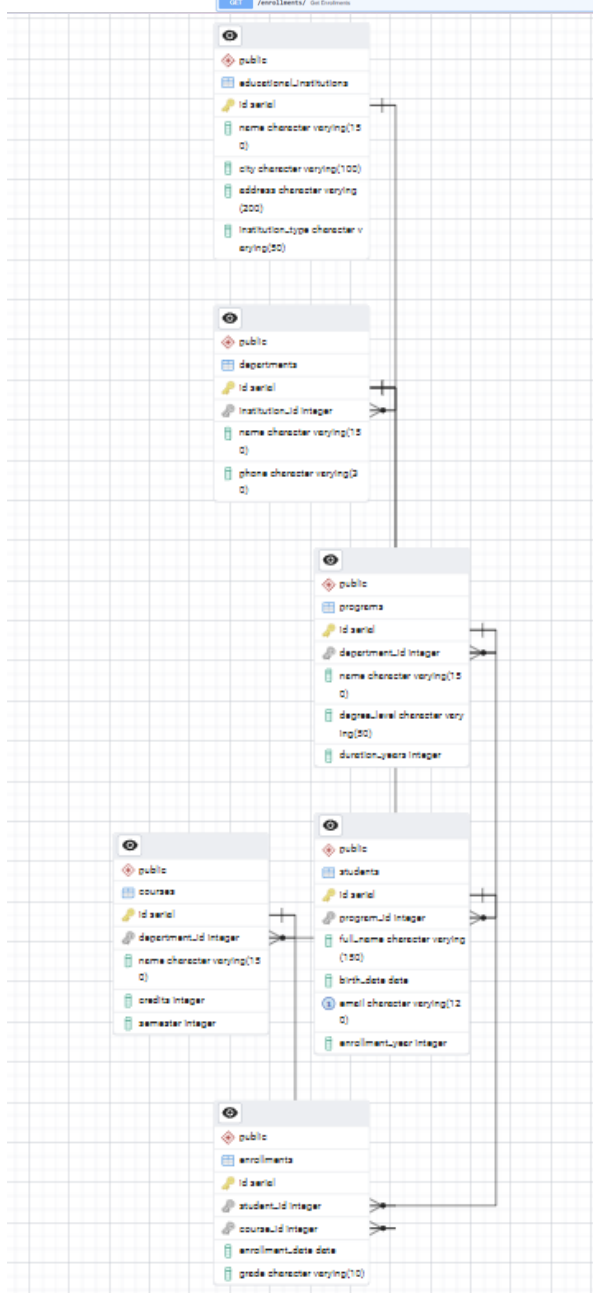
```
"""Get all enrollments."""  
return db.query(Enrollment).all()
```

```
@app.put("/enrollments/{enrollment_id}", response_model=EnrollmentResponse)  
def update_enrollment(  
    enrollment_id: int,  
    enrollment: EnrollmentCreate,  
    db: Session = Depends(get_db),  
) -> Enrollment:  
    """Update enrollment."""  
    found_item = db.query(Enrollment).filter(Enrollment.id == enrollment_id).first()  
    if found_item is None:  
        raise HTTPException(status_code=404, detail="Enrollment not found")  
  
    for key, value in enrollment.model_dump().items():  
        setattr(found_item, key, value)  
  
    db.commit()  
    db.refresh(found_item)  
    return found_item
```

```
@app.delete("/enrollments/{enrollment_id}")  
def delete_enrollment(  
    enrollment_id: int,  
    db: Session = Depends(get_db),  
) -> dict[str, str]:  
    """Delete enrollment."""  
    found_item = db.query(Enrollment).filter(Enrollment.id == enrollment_id).first()  
    if found_item is None:  
        raise HTTPException(status_code=404, detail="Enrollment not found")  
  
    db.delete(found_item)  
    db.commit()  
    return {"message": "Enrollment deleted"}
```

Educational Institutions API GET POST PUT PATCH DELETE

default	^
GET /institutions/	Get Institutions
POST /institutions/	Create Institution
PUT /institutions/{institution_id}	Update Institution
PATCH /institutions/{institution_id}	Partial Update Institution
DELETE /institutions/{institution_id}	Delete Institution
GET /departments/	Get Departments
POST /departments/	Create Department
PUT /departments/{department_id}	Update Department
PATCH /departments/{department_id}	Partial Update Department
DELETE /departments/{department_id}	Delete Department
GET /programs/	Get Programs
POST /programs/	Create Program
PUT /programs/{program_id}	Update Program
PATCH /programs/{program_id}	Partial Update Program
DELETE /programs/{program_id}	Delete Program
GET /students/	Get Students
POST /students/	Create Student
PUT /students/{student_id}	Update Student
PATCH /students/{student_id}	Partial Update Student
DELETE /students/{student_id}	Delete Student
GET /courses/	Get Courses
POST /courses/	Create Course
PUT /courses/{course_id}	Update Course
PATCH /courses/{course_id}	Partial Update Course
DELETE /courses/{course_id}	Delete Course
GET /enrollments/	Get Enrollments



Вывод: приобрёл практические навыки разработки API и баз данных.